

Canteen Management System

DBMS Course Project Report



Student Name: Hansith Raj Kasani

Roll Number: 25WU0102117

Course: Database Management Systems (DBMS)

Academic Year: 2025–2029

Project Type: DBMS Course Project

University: Woxsen University

Branch: CSE - AIML

2. Abstract

The Canteen Management System is a database-driven web application developed to manage menu items, customers, orders, order items, ingredients, recipes, kitchen operations, stock usage, billing, and payments in a structured relational system. The application uses MySQL as the database layer, Python 3 with Flask for backend services, Jinja2 templates and vanilla JavaScript for the user interface, and mysql-connector-python for database connectivity. The system supports menu maintenance, cart-based order entry, kitchen status management, ingredient stock monitoring, billing, payment validation, and management reports. Transaction-based order placement keeps related database writes consistent, while primary keys, foreign keys, validation rules, and database triggers protect data integrity.

3. Introduction and Problem Statement

3.1 Introduction

A canteen produces related data continuously: customers place orders, orders contain multiple food items, food items use ingredients according to recipes, kitchen staff process orders, stock is consumed, bills are generated, and payments settle bills. Managing these records independently can create duplication and inconsistent status or financial information. This project models the complete workflow using a normalized relational database and a connected web interface.

3.2 Problem Statement

The canteen requires a reliable system to maintain menu information, accept and track customer orders, manage kitchen preparation, monitor ingredients, generate bills, record payments, and produce reports. The system must preserve relationships between these records and prevent invalid operations such as ordering unavailable items, using invalid order-status transitions, placing non-positive quantities, or making payments above the bill amount.

4. Objectives and Scope

4.1 Objectives

- Maintain menu categories, food items, recipes, ingredients, customers, kitchen staff, orders, order items, kitchen tasks, stock usage, bills, and payments in MySQL.
- Provide CRUD operations for MenuCategory, FoodItem, and Recipe through the web interface.
- Provide cart-based order entry with live totals and client-side and server-side validation.
- Place an order as an all-or-nothing transaction across Orders, OrderItem, KitchenTask, and StockUsage.
- Manage kitchen preparation using controlled status transitions and staff assignment.
- Monitor ingredient stock, identify low-stock items, and support issue/restock operations.
- Generate bills for served orders and enforce payment limits.
- Provide seven required reports using joins, subqueries, aggregation, and views.

4.2 Scope

- Menu maintenance and searchable food catalog.

- Customer selection and cart-based order placement.
- Kitchen workflow and staff assignment.
- Ingredient stock monitoring and stock usage.
- Billing and payment management.
- Reporting for sales, popular items, category sales, pending orders, ingredient consumption, cancellations, and revenue.
- Local web application deployment using Python, Flask, MySQL, Jinja2, and vanilla JavaScript.

5. Software and Hardware Requirements

Category | Requirement

Operating System | Windows / macOS / Linux

Backend | Python 3 + Flask

Database | MySQL 8.0

DB Connectivity | mysql-connector-python

Frontend | HTML5, CSS, vanilla JavaScript

Templates | Jinja2

Build process | No build step

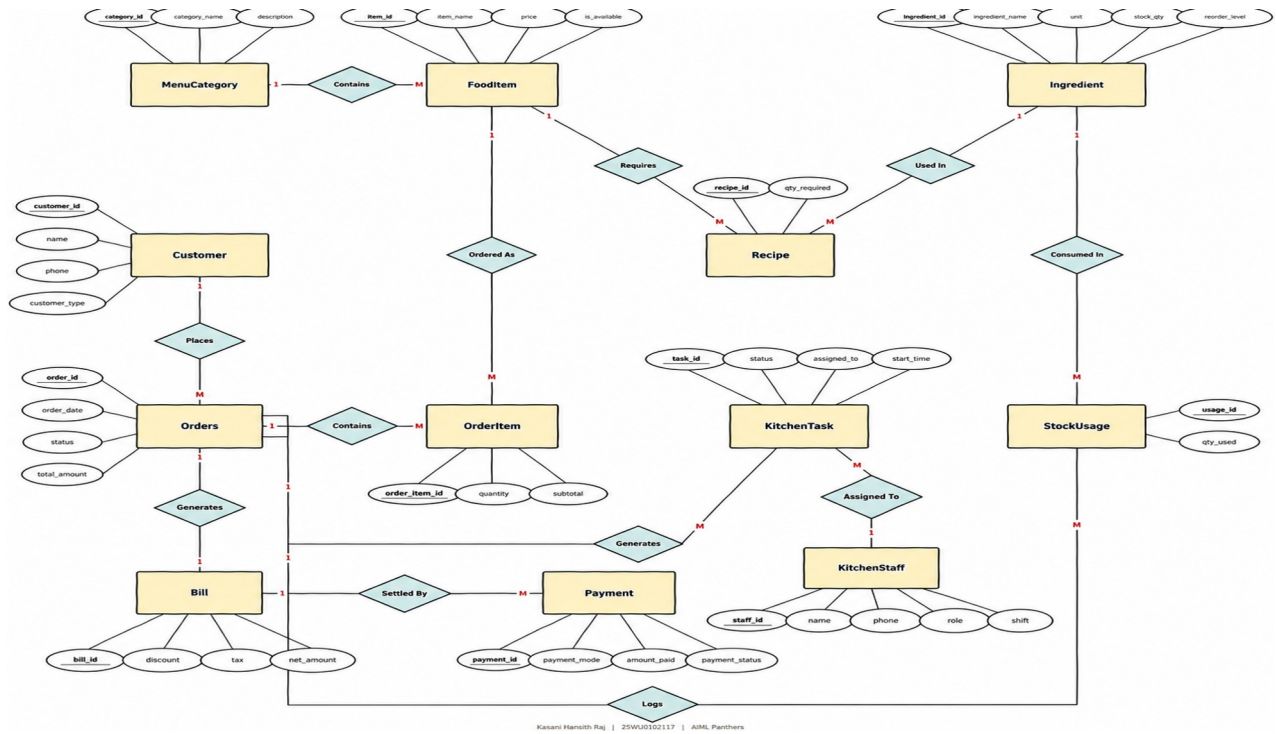
Testing | pytest

Browser | Modern Chromium / Firefox / Safari

Recommended hardware | Dual-core processor, 8 GB RAM, 10 GB free storage

6. ER Diagram

The ER diagram represents the twelve entities used by the canteen system and their relationships.



7. Relational Schema and Normalization

7.1 Relational Schema

7.1 MenuCategory

Field	Type	Null	Key	Default	Extra
category_id	int	NO	PRI	NULL	auto_increment
category_name	varchar(50)	NO	UNI	NULL	
description	varchar(150)	YES		NULL	

7.2 FoodItem

Field	Type	Null	Key	Default	Extra
item_id	int	NO	PRI	NULL	auto_increment
category_id	int	NO	MUL	NULL	
item_name	varchar(80)	NO		NULL	
price	decimal(8,2)	NO		NULL	
is_available	tinyint(1)	NO		1	

7.3 Customer

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
name	varchar(80)	NO		NULL	
phone	varchar(15)	NO	UNI	NULL	
email	varchar(100)	YES		NULL	
customer_type	enum('Student','Staff','Guest')	NO		Student	

7.4 Ingredient

Field	Type	Null	Key	Default	Extra
ingredient_id	int	NO	PRI	NULL	auto_increment
ingredient_name	varchar(80)	NO	UNI	NULL	
unit	varchar(10)	NO		g	
stock_quantity	decimal(10,2)	NO		0.00	
reorder_level	decimal(10,2)	NO		10.00	
cost_per_unit	decimal(10,2)	NO		0.00	

7.5 Recipe

Field	Type	Null	Key	Default	Extra
recipe_id	int	NO	PRI	NULL	auto_increment
item_id	int	NO	MUL	NULL	
ingredient_id	int	NO	MUL	NULL	
qty_required	decimal(8,2)	NO		NULL	

7.6 KitchenStaff

Field	Type	Null	Key	Default	Extra
staff_id	int	NO	PRI	NULL	auto_increment
name	varchar(80)	NO		NULL	
phone	varchar(15)	NO	UNI	NULL	
role	enum('Head Cook','Cook','Helper')	NO		Cook	
shift	enum('Morning','Afternoon','Evening')	NO		Morning	

7.7 Orders

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	auto_increment
customer_id	int	NO	MUL	NULL	
order_date	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
status	enum('Placed','Preparing','Ready','Served','Cancelled')	NO		Placed	
total_amount	decimal(10,2)	NO		0.00	

7.8 OrderItem

Field	Type	Null	Key	Default	Extra
order_item_id	int	NO	PRI	NULL	auto_increment
order_id	int	NO	MUL	NULL	
item_id	int	NO	MUL	NULL	
quantity	int	NO		NULL	
unit_price	decimal(8,2)	NO		NULL	
subtotal	decimal(10,2)	NO		NULL	

7.9 KitchenTask

Field	Type	Null	Key	Default	Extra
task_id	int	NO	PRI	NULL	auto_increment
order_id	int	NO	MUL	NULL	
staff_id	int	YES	MUL	NULL	

status	enum('Pending','Cooking','Ready','Served')	NO		Pending	
start_time	datetime	YES		NULL	
end_time	datetime	YES		NULL	

7.10 StockUsage

Field	Type	Null	Key	Default	Extra
usage_id	int	NO	PRI	NULL	auto_increment
ingredient_id	int	NO	MUL	NULL	
order_id	int	NO	MUL	NULL	
qty_used	decimal(8,2)	NO		NULL	
usage_date	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

7.11 Bill

Field	Type	Null	Key	Default	Extra
bill_id	int	NO	PRI	NULL	auto_increment
order_id	int	NO	UNI	NULL	
bill_date	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
discount	decimal(8,2)	NO		0.00	
tax	decimal(8,2)	NO		0.00	
net_amount	decimal(10,2)	NO		NULL	

7.12 Payment

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	auto_increment
bill_id	int	NO	MUL	NULL	
payment_mode	enum('Cash','Card','UPI','Waller')	NO		NULL	
amount_paid	decimal(10,2)	NO		NULL	
payment_date	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
payment_status	enum('Paid','Failed','Refunded')	NO		Paid	

Note

The tables above reproduce the actual MySQL DESC structure of the implemented database, including Field, Type, Null, Key, Default, and Extra values.

7.2 Normalization

The schema follows 1NF, 2NF, and 3NF principles. Recipe resolves the many-to-many FoodItem–Ingredient relationship, while OrderItem represents individual order lines.

9.1 Database and table creation

```
CREATE DATABASE IF NOT EXISTS CanteenManagement;
USE CanteenManagement;
```

```
CREATE TABLE MenuCategory (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL,
    description VARCHAR(255)
```

```
);

CREATE TABLE FoodItem (
    item_id INT PRIMARY KEY AUTO_INCREMENT,
    category_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    is_available BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (category_id) REFERENCES MenuCategory(category_id)
);
```

9.2 Representative DML

```
INSERT INTO MenuCategory(category_name, description)
VALUES ('Main Course', 'Main meals');
```

```
UPDATE FoodItem SET is_available = FALSE WHERE item_id = 5;
```

```
DELETE FROM Customer WHERE customer_id = 20;
```

9.3 Sample Outputs

SHOW TABLES; → All 12 canteen tables are listed.

SELECT * FROM Ingredient; → Ingredient records are displayed with stock and cost fields.

SELECT * FROM Orders; → Order records are displayed with status and total amount.

9. Queries with Outputs, including Presentation-II Query

9.1 Total Amount for Each Category

```
SELECT
    m.category_name,
    SUM(oi.subtotal) AS total_amount
FROM MenuCategory m
JOIN FoodItem f
    ON m.category_id = f.category_id
JOIN OrderItem oi
    ON f.item_id = oi.item_id
JOIN Orders o
    ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY m.category_id, m.category_name;
```

Actual output:

Category	Total Amount
Breakfast	1580.00
Beverages	400.00
Snacks	90.00
Main Course	310.00
Desserts	30.00
South Indian	205.00
North India	730.00
Chinese	680.00
Fast Food	130.00
Rice Items	70.00
Curries	75.00
Juices	100.00

9.2 Number of Orders Placed by Each Customer

```
SELECT
    c.name AS customer_name,
```

```

COUNT(o.order_id) AS total_orders
FROM Customer c
LEFT JOIN Orders o
      ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;

```

Actual output:

Customer Name	Total Orders
Arjun Rao	15
Priya Nair	1
Dr. Nagaraju D	1
Kiran Kumar	1
Meera Iyer	1
Rohan Das	1
Aditya Sharma	1
Meghana Rao	1
Rahul Verma	1
Sneha Reddy	1
Vikram Singh	1
Ananya Gupta	1
Karthik Rao	1
Priya Menon	1
Rohit Kumar	1
Ishita Patel	1
Nikhil Reddy	1
Aarav Mehta	1
Divya Sharma	1
Manish Rao	1
Neha Kapoor	1
Raj	2

9.3 Most Popular Food Item

```

SELECT
    f.item_name,
    SUM(oi.quantity) AS quantity_sold
FROM OrderItem oi
JOIN FoodItem f
      ON oi.item_id = f.item_id
JOIN Orders o
      ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY f.item_id, f.item_name
ORDER BY quantity_sold DESC
LIMIT 1;

```

Actual output:

Item Name	Quantity Sold
Masala Dosa	21

9.4 Presentation-II Query — Net Profit of Total Canteen

The calculation is shown in three steps: Total Revenue, Total Ingredient Cost, and Net Profit.

Step 1: Total Revenue

```

SELECT
    SUM(oi.subtotal) AS total_revenue
FROM OrderItem oi
JOIN Orders o
      ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled';

```

Total Revenue
44000.00

Step 2: Total Ingredient Cost

```
SELECT
    SUM(
        oi.quantity *
        r.qty_required *
        i.cost_per_unit
    ) AS total_ingredient_cost
FROM OrderItem oi
JOIN Orders o
    ON oi.order_id = o.order_id
JOIN Recipe r
    ON oi.item_id = r.item_id
JOIN Ingredient i
    ON r.ingredient_id = i.ingredient_id
WHERE o.status <> 'Cancelled';
```

Total Ingredient Cost
1304.8500

Step 3: Total Net Profit

```
SELECT
    SUM(oi.subtotal)
    -
    SUM(
        oi.quantity *
        r.qty_required *
        i.cost_per_unit
    ) AS net_profit
FROM OrderItem oi
JOIN Orders o
    ON oi.order_id = o.order_id
JOIN Recipe r
    ON oi.item_id = r.item_id
JOIN Ingredient i
    ON r.ingredient_id = i.ingredient_id
WHERE o.status <> 'Cancelled';
```

Net Profit
7295.1500

Net Profit = Total Revenue – Total Ingredient Cost = 44000.00 – 1304.8500 = 7295.1500.

10. UI Design and Screenshots

The following screenshots are from the implemented CanteenHub application.

Figure 1. CanteenHub Dashboard

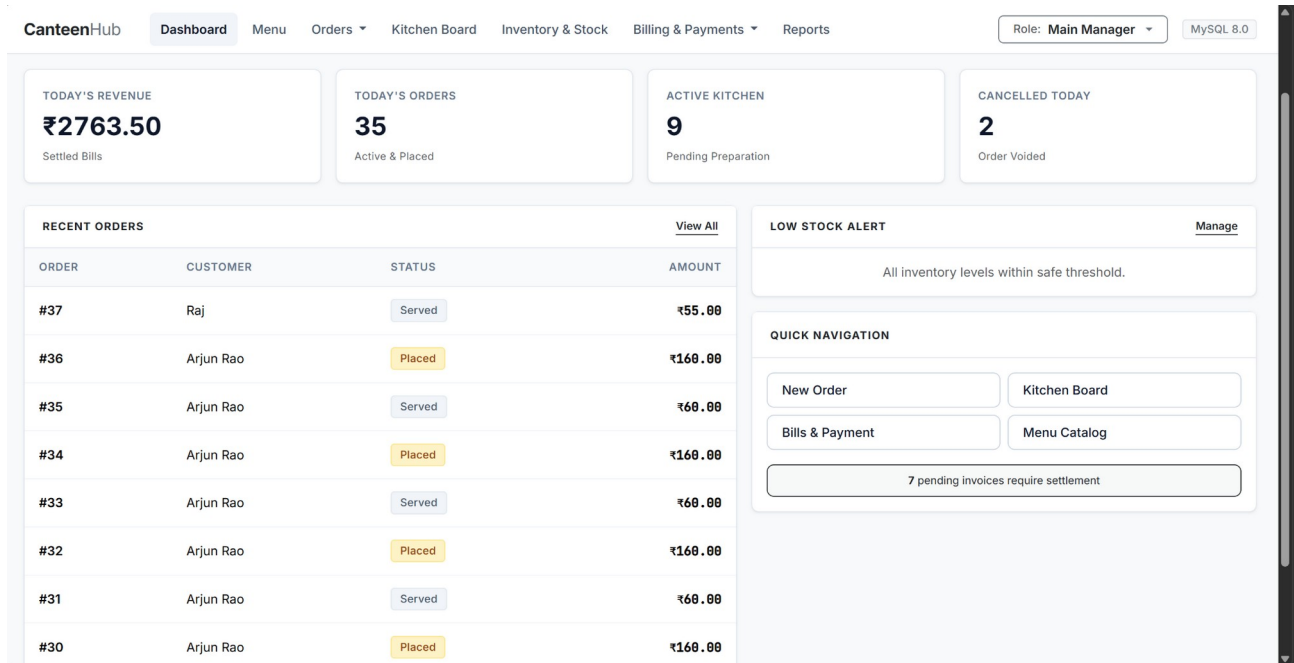


Figure 2. Reports Dashboard

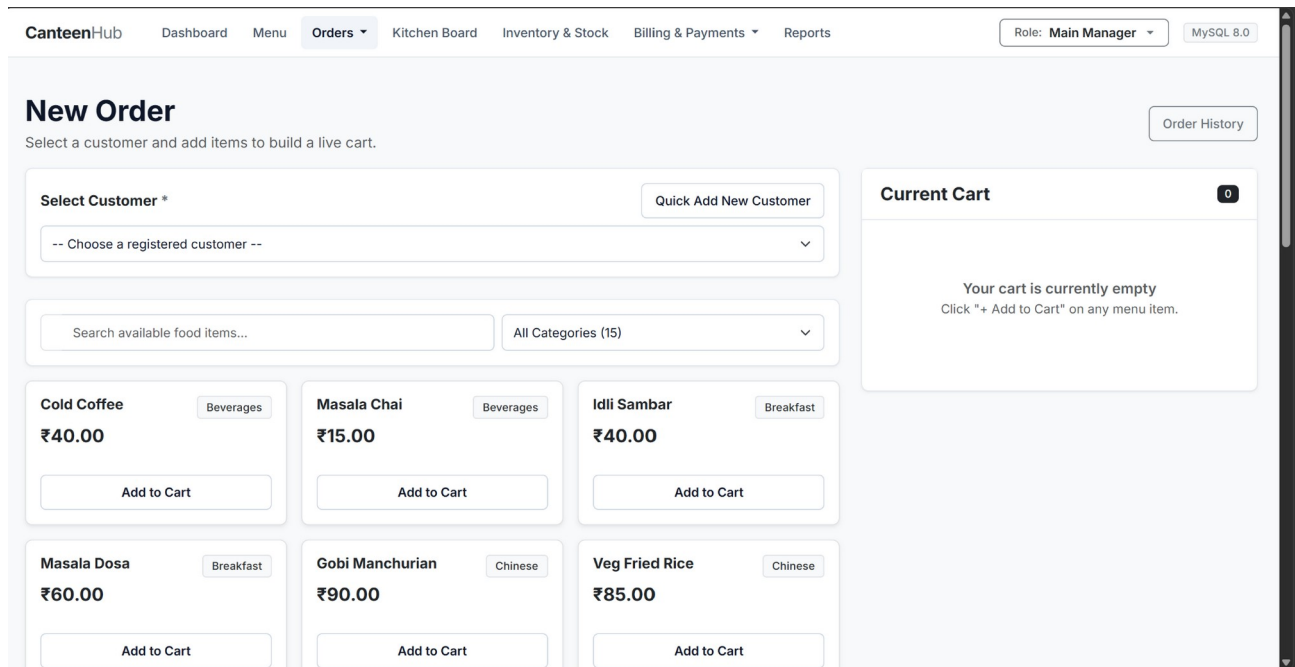


Figure 4. Additional CanteenHub Application Screen

11. Implementation Details

12.1 Technology Stack

Layer | Technology | Purpose

Backend | Python 3 + Flask | Routes, business logic, validation and transactions

Templates | Jinja2 | Server-rendered HTML

Frontend | HTML5 + CSS + vanilla JavaScript | UI, cart, live totals and client validation

Database | MySQL 8.0 | Persistent relational data

DB Driver | mysql-connector-python | MySQL connectivity

Testing | pytest | Automated tests against seeded sample data

Build | No build step | Direct Flask/Jinja/JS execution

12.2 DB Connectivity and Transaction Pattern

```
@contextmanager
def transaction():
    conn = get_connection()
    try:
        yield conn
```

```

        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

```

Every multi-table write uses `transaction()`. Order placement treats Orders, OrderItem, KitchenTask and StockUsage as one transaction.

12.3 Validation and Error Handling

- Client-side and server-side validation for every form.
- Unavailable food items are rejected.
- Positive quantities are required.
- Order status transitions are controlled.
- Payment cannot exceed bill net amount.
- Database errors are logged server-side and converted to plain-English browser messages.

12.4 Project Structure

```

project/
├── app.py
├── db.py
├── requirements.txt
├── PROJECT_BRIEF.md
├── TEST_PLAN.md
├── db/schema.sql
├── db/sample_data.sql
├── templates/
├── static/
├── tests/
└── screenshots/

```

12. Testing (Test Cases and Results)

Database connection | Application connects to MySQL | To be verified

Menu CRUD | Records persist correctly | To be verified

Order transaction | All related writes commit together | To be verified

Order rollback | Failed write rolls back | To be verified

Kitchen transitions | Only valid transitions succeed | To be verified

Low-stock detection | Low-stock items are flagged | To be verified

Billing/payment | Billing and payment limits work | To be verified

Reports | Seven required reports return correct results | To be verified

Error handling | Raw errors do not reach browser | To be verified

UI navigation | Major screens load correctly | To be verified

13. Conclusion and Future Enhancements

14.1 Conclusion

The Canteen Management System provides a normalized relational solution for menu, customer, order, kitchen, inventory, billing, payment and reporting workflows.

14.2 Future Enhancements

- Authentication and role-based access control.
- Real-time kitchen notifications.
- Online payment gateway integration.
- Automated inventory/reorder management.
- Advanced analytics.
- Cloud deployment and backups.

14. References

- MySQL Documentation.
- Python 3 Documentation.
- Flask Documentation.
- mysql-connector-python Documentation.
- Reference DBMS Course Project Report supplied for the report structure.

15. Appendix: GitHub Repository Link

GitHub Repository: _____

Insert the final repository URL before submission.